P6 evaluation handoff

Updated: 5 November 2026.

This is an **offline scoring package**. It does not run RAG. Each team owner runs their own system and saves the outputs, then the evaluation lead turns those files into report tables. **The evaluation lead is not decided yet, and it won't be P6.**

```
Owners run static / adaptive RAG → save answers, contexts, timings
        → evaluation lead scores them against gold → tables for the report
```

Ground rules:

- Run every command from the repository root.
- Raw outputs stay local (`results/` is ignored by Git). Commit only what section 11 lists. Never commit model outputs, judge traces, credentials, corpora or indexes.
- Each command writes to a **new** output directory and refuses to overwrite. Pick a fresh name each run.
- Schemas and split mappings are still `v0-proposed`.
- Metric definitions are in EVALUATION_METRICS_REFERENCE.md.

1. Who does what

These owners are unknown. P6 will not run the end-to-end evaluation. Fill this in before the final run; each role needs one named person.

Role	Name	Responsible for
Evaluation lead	*unknown*	Runs steps 3–10, checks results, commits the summaries
Batch operator	*unknown*	Runs the final static and adaptive batches and delivers the files in section 1
Judge owner	*unknown*	Gets the approved judge model, API key and budget, and runs step 7
Report owner	*unknown*	Puts the tables in the report with the required disclosures
Module owners	*unknown*	Review any claim about their module before it goes in the report

The whole team confirms the question freeze, independence and the use of `draft-v3` as the gold before anything is reported as final.

Status

Step	Owner	Status	Blocked on
3a Audit + export P3 inputs	Done	✅ Done (`results/p6/p3-snapshot-20261105/`)	–
3b–3d Review files + final gold	Unknown	⏳ Draft ready (`draft-v3`)	Team confirms annotator independence, question freeze and role
4 Retrieval scores	Unknown	⏳ Development run done	Final gold (3c)
5 QA scores	Unknown	⏳ To do	P4/P5 answer files
6 Evidence recall	Unknown	⏳ To do	P4 static top-10 + adaptive records
7 RAGAS	Unknown	⏳ To do	Exact contexts, approved judge model + API key
8 Performance/cost	Unknown	⏳ To do	P4/P5 timings, usage, batch wall time
9–12 Comparisons, saving, checks, report tables	Unknown	⏳ To do	Steps 4–8

Open team decisions: who runs the final batch, which judge model, and the judge budget.

Files you need

The gold labels are in Git at `data/gold/p6/draft-v3/`. The files below are not on this branch, so put each one at the listed path.

File	Path	Where it comes from	Local now?
Corpus (28,617 passages, all paper text)	`data/processed/passages_all_paper_text.jsonl` + `.manifest.json`	P3 commit, see command below	✅
QASPER QA (references)	`data/processed/qa.jsonl` + `qasper_export.manifest.json`	P3 commit	✅
Evidence mapping + manifest	`data/processed/gold_evidence_mapping_all_splits.{jsonl,manifest.json}`	P3 commit	✅
P3 frozen inputs	`results/p6/p3-snapshot-20261105/`	Already exported (step 3a)	✅

File	Path	Where it comes from	Local now?
Annotation audit	`results/p6/annotation-audit-verified-20261105/`	Already exported (step 3a)	✅
P4/P5 answer files	`results/p4-final/` (create it)	P4/P5 deliver them, see checklist below	❌

All P3 data files are committed at P3 commit `1a1dc15b`, so you don't need to download anything. On a new machine, extract each one like this:

```
for f in passages_all_paper_text.jsonl
passages_all_paper_text.manifest.json qa.jsonl qasper_export.manifest.json
\
         gold_evidence_mapping_all_splits.jsonl
gold_evidence_mapping_all_splits.manifest.json; do
  git show 1a1dc15b:data/processed/$f > data/processed/$f
done
```

Use `passages_all_paper_text.jsonl`, **not** `passages.jsonl`. All 1,853 pool passages match the all-paper-text corpus, but only 703 match `passages.jsonl`. The wrong file fails with "Corpus identity/text differs from the frozen pool". Expected SHA-256 prefixes: corpus `8eeef011…`, `qa.jsonl` `1515fc58…`, mapping `6bfd323a…`.

What P4/P5 must deliver

Put these in `results/p4-final/`, as one JSONL row per question, covering every question in `p3-snapshot-20261105/questions.jsonl`:

- ☐ `static.jsonl` and `adaptive.jsonl`: agent records from `scripts/run_agent_experiment.py`. Each row needs `question_id`, an answer `status`, `retrieval_log`, and timings `elapsed_s` + `answer_s`.
- ☐ `static-top10.jsonl`: a static run whose saved ranking is at least 10 deep (for evidence recall @5/@8/@10).
- ☐ `static-contexts.jsonl` / `adaptive-contexts.jsonl`: the exact text shown to the generator (format in step 7a), **or** the original `PromptConfig` JSON so the evaluation lead can rebuild it.
- ☐ `run-notes.md`: generator model, prompt version, hardware, concurrency, total batch wall-clock seconds, and token usage per call if available.

2. Setup and a practice run

```
export P6_PY="$PWD/.venv-p6/bin/python"
"$P6_PY" -m scripts.run_p6_evaluation --help
```

No `.venv-p6` yet? Create any Python 3.10+ environment and run `pip install -r requirements-dev.txt`, then point `P6_PY` at its Python.

Try it on the bundled **synthetic** files first (these numbers mean nothing for the project):

```
"$P6_PY" -m scripts.run_p6_evaluation qa \
  --questions tests/fixtures/p6/questions.jsonl \
  --predictions tests/fixtures/p6/static.jsonl \
  --references tests/fixtures/p6/references.jsonl \
  --arm static --role development \
  --output results/p6/example-static-qa
```

You should get `qa.csv`, `scores.jsonl`, `summary.json` and `run_manifest.json`, with answerable F1 = 0.8. Repeat with `adaptive.jsonl`, `--arm adaptive` and output `results/p6/example-adaptive-qa` (F1 = 1.0).

Set these once per shell for the real steps below:

```
export P3_COMMIT=1a1dc15b824590a39c80ea11a568fbe5814a926e
export P6_CORPUS="$PWD/data/processed/passages_all_paper_text.jsonl"   # the corpus the pool was built from
export P6_QUESTIONS="$PWD/results/p6/p3-snapshot-20261105/questions.jsonl"
export P6_REFERENCES="$PWD/data/processed/qa.jsonl"
```

3. Build the gold labels (done once)

Step 3a – audit annotations and export P3 inputs (✅ already done, skip). The outputs are in `results/p6/annotation-audit-verified-20261105/` and `results/p6/p3-snapshot-20261105/`. Rerun only if P3 changes the commit, and write to new directories:

```
"$P6_PY" -m scripts.run_p6_evaluation audit-labels \
  --ref "$P3_COMMIT" --output results/p6/annotation-audit-<date>

"$P6_PY" -m scripts.run_p6_evaluation export-p3 \
  --ref "$P3_COMMIT" --output results/p6/p3-snapshot-<date>
```

✔ Done when: the audit shows 1,853 pairs, 200 questions, 89.854% raw agreement and 188 disagreements, and the snapshot has `questions.jsonl`, `pairs.jsonl` and `rankings.jsonl`.

Step 3b – write the review file `results/p6/annotation-review.json`. Fill it in only after the team has actually confirmed these points:

```
{
  "source_commit": "1a1dc15b824590a39c80ea11a568fbe5814a926e",
  "independence_confirmed": true,
  "usage_and_freeze_reviewed": true,
  "project_role": "held_out",
  "review_note": "Who confirmed what, and when"
}
```

Use `"development"` as the role only if the questions really are development-only.

Step 3c – finalize gold. The 188 disagreements have final labels, recorded in `data/gold/p6/draft-v3/resolved_labels.csv`. Add a `resolution_disclosure` statement to the review file, then run:

```
"$P6_PY" -m scripts.run_p6_evaluation finalize-labels \
  --ref "$P3_COMMIT" --disagreements resolved \
  --adjudication data/gold/p6/draft-v3/resolved_labels.csv \
  --review results/p6/annotation-review.json \
  --corpus "$P6_CORPUS" \
  --output results/p6/final-gold
```

✔ Done when: `results/p6/final-gold/` has `final_labels.jsonl` and `final_labels_manifest.json`, with 200 questions, 1,853 judged pairs, 820 relevant pairs and `annotation_coverage: "complete"`.

This is not third-person adjudication. Any report using this gold must say that one reviewer resolved the 188, that this reviewer annotated 73 of them, that 66 labels are unchecked AI suggestions, that AI suggestions may have been visible during review, and that most records have no written reason. If the team later gets third-person decisions, use `--disagreements adjudicate` instead.

Current draft: `data/gold/p6/draft-v3/` is **not team-confirmed** and has role `development`. It uses `results/p6/gold-draft-v3-review.json`, whose confirmation flags are placeholders. Regenerate it after sign-off and don't report its scores as final. Put the final gold in `data/gold/p6/` too.

Step 3d – final evaluation review (needed whenever `--role held_out`). This creates `results/p6/final-evaluation-review.json` with all question IDs filled in:

```
"$P6_PY" - <<'PYCODE'
import json
from pathlib import Path
qs = [json.loads(l) for l in Path('results/p6/p3-snapshot-
20261105/questions.jsonl').read_text().splitlines() if l]
with Path('results/p6/final-evaluation-review.json').open('x') as f:
    json.dump({'usage_and_freeze_reviewed': None, 'review_note': '',
               'question_ids': [q['question_id'] for q in qs]}, f,
indent=2)
PYCODE
```

Once the team has frozen the model, prompts, index and settings, set `usage_and_freeze_reviewed` to `true` and write a note pointing to that freeze record.

> For development-only runs in the steps below, use `--role development` and drop `--review`.

4. Score retrieval

Scores the saved top-5 rankings of all four methods. No model is rerun.

```
"$P6_PY" -m scripts.run_p6_evaluation retrieval \
  --questions "$P6_QUESTIONS" \
  --pairs results/p6/p3-snapshot-20261105/pairs.jsonl \
  --rankings results/p6/p3-snapshot-20261105/rankings.jsonl \
  --labels results/p6/final-gold/final_labels.jsonl \
  --labels-manifest results/p6/final-gold/final_labels_manifest.json \
  --corpus "$P6_CORPUS" --ks 1 3 5 \
  --role held_out --review results/p6/final-evaluation-review.json \
  --output results/p6/retrieval-final
```

✔ Done when: `retrieval.csv` has one row per method (BM25, Dense, Hybrid RRF, Hybrid RRF + rerank) $\times K$, 12 rows in all, with 200 questions and `judged_coverage` of 1.0.

5. Score QA answers

Each prediction row needs `question_id` and `status` (`answered`, `refused` or `error`). P4 and P5 output formats are read directly. If a file only has `request_id`, ask its owner for a verified ID mapping. Never match rows by position.

```
export P6_PREDICTIONS="$PWD/results/p4-final/static.jsonl"

"$P6_PY" -m scripts.run_p6_evaluation qa \
  --questions "$P6_QUESTIONS" --predictions "$P6_PREDICTIONS" \
  --references "$P6_REFERENCES" --arm static \
  --role held_out --review results/p6/final-evaluation-review.json \
  --output results/p6/static-qa-final
```

Run it again for adaptive (`--arm adaptive`, new output directory). Missing predictions count as failures. Report answerable EM/F1 and unanswerable correctness separately.

✔ Done when: `qa.csv` exists for both arms, and its question count matches `questions.jsonl` (200). Most rows should be `answered` or `refused`; a high `error` count means you should go back to P4/P5.

6. Mapped evidence recall (static @5/@8/@10 vs adaptive)

This uses P3's evidence mapping, not the pool labels. Static needs a saved ranking at least as deep as the largest K. Adaptive takes the union of passages over all of its rounds.

```
export
P6_MAPPING="$PWD/data/processed/gold_evidence_mapping_all_splits.jsonl"
export
P6_MAPPING_MANIFEST="$PWD/data/processed/gold_evidence_mapping_all_splits.m
anifest.json"

"$P6_PY" -m scripts.run_p6_evaluation evidence \
  --questions "$P6_QUESTIONS" --records results/p4-final/static-top10.jsonl
\
  --gold "$P6_MAPPING" --gold-manifest "$P6_MAPPING_MANIFEST" --corpus
"$P6_CORPUS" \
  --mode static --arm static --saved-depth 10 --ks 5 8 10 \
  --role held_out --review results/p6/final-evaluation-review.json \
  --output results/p6/static-evidence

"$P6_PY" -m scripts.run_p6_evaluation evidence \
  --questions "$P6_QUESTIONS" --records results/p4-final/adaptive.jsonl \
  --gold "$P6_MAPPING" --gold-manifest "$P6_MAPPING_MANIFEST" --corpus
"$P6_CORPUS" \
  --mode adaptive --arm adaptive \
  --role held_out --review results/p6/final-evaluation-review.json \
  --output results/p6/adaptive-evidence
```

✔ Done when: `evidence_recall.csv` exists for both arms, and static has rows for $K = 5, 8, 10$. To match P4's own numbers, use `successful_only_mapped_recall`. Never show mapping rows to the model.

7. RAGAS (LLM judge)

7a – get the exact context. RAGAS needs the passages the generator actually saw. The best option is for P4/P5 to export it directly:

```
{"question_id":"q1","paper_id":"paper1","exact_rendered":true,"contexts":["
[1] passage_id: p3\n    text: ..."],"provenance":
{"run":"...","method":"captured from the answer prompt"}}
```

If they can't, rebuild it from the agent records plus P5's original `PromptConfig`, saved as `results/p6/prompt-config.json`:

```
"$P6_PY" -m scripts.run_p6_evaluation contexts \
  --predictions "$P6_PREDICTIONS" \
  --prompt-config results/p6/prompt-config.json \
  --expect-prompt-version p5-grounded-qa-v7 \
  --review-note 'Original run/config reference' \
  --output results/p6/static-contexts
```

7b – dry run (free). Leave out `--live`. This checks the inputs without calling the judge, so scores come back as `ready_not_scored`. Try it on the synthetic files (`tests/fixtures/p6/contexts.jsonl`) first, then on the real files.

7c – judge environment (separate from the vLLM environment):

```
python3.12 -m venv .venv-p6-judge
.venv-p6-judge/bin/python -m pip install -r requirements-evaluation.txt
.venv-p6-judge/bin/python -m pip freeze > results/p6/judge-dependencies.txt
```

Create `results/p6/judge-config.json`. It accepts any OpenAI-compatible chat + embedding endpoint, and you can add `base_url` to choose one:

```
{
  "judge_model": "APPROVED_MODEL",
  "judge_revision": "VERSION_NOTE",
  "embedding_model": "APPROVED_EMBEDDING_MODEL",
  "embedding_revision": "VERSION_NOTE",
  "api_key_env": "P6_JUDGE_API_KEY",
  "timeout_seconds": 120
}
```

Put the key in the `P6_JUDGE_API_KEY` environment variable, never in a file.

7d – live run. Test on a small development set first, then do the final run:

```
.venv-p6-judge/bin/python -m scripts.run_p6_evaluation ragas \
  --questions "$P6_QUESTIONS" --predictions "$P6_PREDICTIONS" \
  --references "$P6_REFERENCES" \
  --contexts results/p6/static-contexts/contexts.jsonl \
  --arm static --role held_out --review results/p6/final-evaluation-
review.json \
  --live --judge-config results/p6/judge-config.json \
  --cache results/p6/judge-cache \
  --output results/p6/static-ragas-final
```

✔ Done when: `ragas.csv` has non-null scores for most questions. It reports faithfulness, response relevance and context precision. Report valid-score counts, not just means. If some calls fail, rerun into a new output directory; scores that already succeeded are reused from the cache. Note that the live judge has **not** been tested against a real endpoint yet.

8. Performance and cost

```
"$P6_PY" -m scripts.run_p6_evaluation performance \
  --predictions "$P6_PREDICTIONS" --arm static \
```

```
  --latency-field internal_pipeline_s --batch-seconds <measured wall time>
\
  --hardware "<GPU/CPU>" --concurrency 1 --cache-policy no-result-cache \
  --usage <usage.jsonl> --pricing <pricing.json> \
  --output results/p6/static-performance
```

- `--batch-seconds` is the measured wall-clock time for the whole batch. If you don't have it, leave it out and throughput will be null. Don't add up per-request times instead.
- Usage rows look like `{"component":"generator","model":"...","input_tokens":1000,"output_tokens":100}`. Pricing is keyed by model. Unknown prices give a null cost.
- See `tests/fixtures/p6/usage.jsonl` and `pricing.json` for the format.
- Take the batch time, hardware and concurrency from P4/P5's `run-notes.md`. Use the same values for both arms.

✔ Done when: `performance.csv` exists for both arms, with latency p50/p95 and sample counts. Throughput and cost are filled in only where real measurements were supplied.

9. Compare two runs

```
"$P6_PY" -m scripts.run_p6_evaluation compare \
  --left results/p6/static-qa-final/scores.jsonl \
  --right results/p6/adaptive-qa-final/scores.jsonl \
  --metrics f1 task_score --resamples 1000 --seed 42 \
  --output results/p6/static-vs-adaptive-qa
```

✔ Done when: `ablation.csv` lists each metric with its difference and confidence interval. Differences are **right minus left**, with paper-level bootstrap confidence intervals.

- Retrieval: pass the same `scores.jsonl` as both left and right, and add `--left-arm A --right-arm B --k 5`.
- Evidence: add `--left-k 5` (or 8, or 10) for the static side and use `--metrics mapped_recall`.

For a fair comparison, keep the generator, prompt, questions, corpus and context budget the same on both sides.

10. What to expect and how to read it

Output is tables, not graphs. Each step writes CSV and JSON files. Nothing draws charts, so the report owner makes any figures from the CSVs. The most useful are: Recall@K by method, static vs adaptive F1 with confidence intervals, and latency p50/p95.

Reference scores. Development run on `draft-v3`, $K = 5$. Your retrieval numbers should match these exactly unless the gold changes:

Method	Precision@5	Pool recall@5	MRR@5	Pool nDCG@5
Hybrid RRF	0.539	0.713	0.804	0.731

Method	Precision@5	Pool recall@5	MRR@5	Pool nDCG@5
Dense	0.527	0.697	0.799	0.718
Hybrid RRF + rerank	0.509	0.658	0.717	0.663
BM25	0.482	0.606	0.729	0.635

There are no reference scores for QA, RAGAS or performance yet. The bundled F1 values of 0.8 and 1.0 are synthetic and aren't targets.

Reading the metrics:

Metric	Plain meaning	Watch out for
Precision@K	Share of the top K that is relevant	Limited by how many relevant passages a question has
Pool recall@K	Share of known relevant passages found in the top K	"Known" means found by these four methods, so it overstates true recall
MRR@K	How high the first relevant passage ranks	Only looks at the first hit
Pool nDCG@K	Ranking quality, rewarding relevant passages near the top	Also limited to the pool
Answerable EM / F1	Exact and word-overlap match with the reference answer	Report separately from unanswerable correctness
Evidence recall	Share of mapped evidence that was retrieved	Static uses a fixed K; adaptive uses all rounds, so it can see more passages
Faithfulness	Whether the answer is supported by its contexts	LLM judgment; report valid-score counts
Latency / cost	Speed and spend	Only comparable with the same hardware, concurrency and caching

Comparing two systems. In `ablation.csv`, differences are right minus left. If the confidence interval includes 0, say there is no clear difference, even if one mean is higher. If two systems differ only slightly, treat them as tied.

Reporting rules:

- Never change a model, prompt or setting because of held-out results. That would stop them being held-out.
- Report surprising results as they are. For example, reranking currently scores below plain Hybrid RRF, and that needs an explanation rather than a rerun.
- Always give question counts, refusals, errors and missing predictions next to the means.
- Include the gold disclosure from step 3c.

11. Save, commit and check results

> Proposed storage rule; the team should confirm it before the final run.

What	Where	In Git?
Full output folders, including `scores.jsonl`	`results/p6/<run-name>/`	No; share through team storage
Final gold labels, manifest and `resolved_labels.csv`	`data/gold/p6/`	Yes
Summary tables (`retrieval.csv`, `qa.csv`, `evidence_recall.csv`, `ragas.csv`, `performance.csv`, `ablation.csv`) and each `run_manifest.json`	`docs/P6/results/<date>-<run-name>/`	Yes
Predictions, contexts, judge traces, API keys	Local only	Never

Commit on your own task branch and open a pull request, following `CONTRIBUTING.md`. Name the commit after the run and say whether it is development or held-out.

Check before committing:

- ☐ `"$P6_PY" -m pytest tests/test_p6_evaluation.py -q` passes.
- ☐ Each step's ✔ condition is met.
- ☐ Question counts are 200, or missing questions are explained.
- ☐ Retrieval scores match section 10, unless the gold changed.
- ☐ `run_manifest.json` shows the commit you expected, and final runs use `--role held_out`.
- ☐ Rerunning `compare` with the same seed gives the same `ablation.csv`.
- ☐ Both arms used the same questions, corpus, generator and context budget.
- ☐ A second person has read the tables, and module owners have reviewed claims about their modules.

12. What goes into the report

Output	Report section
Annotation agreement (`annotation-audit/`)	Q4.3
`retrieval.csv`	Q3
`qa.csv`	Q4.4
`ragas.csv` + judge config	Q4.5
`performance.csv` + cost	Q2.2 / Q4.6
`ablation.csv`	Q5

13. Common errors

Error	Fix
Output already exists	Use a new output directory
Corpus/hash mismatch	Check you're using `passages_all_paper_text.jsonl`, then get the exact original file from its owner. Never edit hashes
Review rejected	Fill in the review JSON with real confirmations. Placeholders fail on purpose
Predictions only have `request_id`	Ask the owner for a verified question-ID mapping
Null throughput/cost	Provide the measured batch time, real usage and prices
RAGAS import/backend error	Use `.venv-p6-judge`. Scoring without the judge doesn't need it

Run the tests:

```
"$P6_PY" -m pytest tests/test_p6_evaluation.py -q
```

Scoring code lives in `src/evaluation/`, and the CLI is `scripts/run_p6_evaluation.py`.